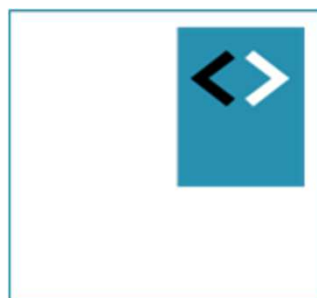




TBWB

Angular Fundamentals

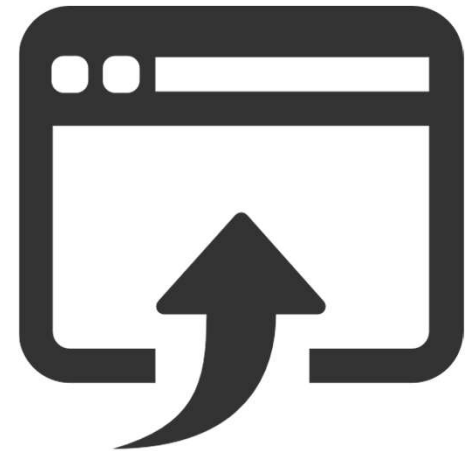
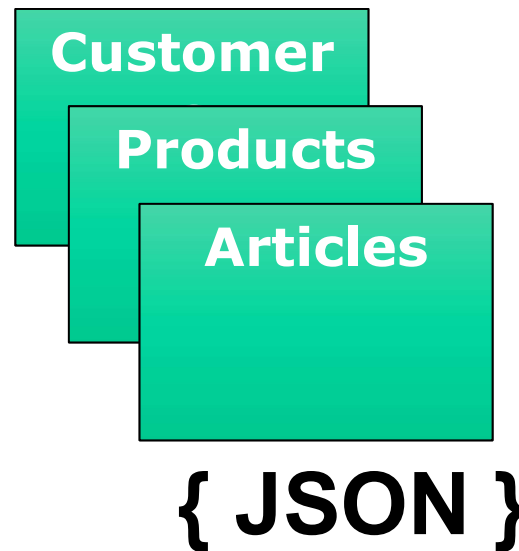
Module - Databinding



Peter Kassenaar –
info@kassenaar.com

Wat is databinding

- Show – all kinds of – data in User Interface
- Data can come from:
 - Controller / class
 - Database
 - User input
 - Other systems



Declarative syntax

- Four (4) kinds of databinding
- Angular specific notation in HTML templates
 1. Simple data binding
 2. Event binding
 3. One-way data binding (Attribute binding)
 4. Two-way data binding



1. Simple Data binding

Class-properties binden in de template

Variables: wrap them in a `signal()`

- Not mandatory – but best practice for reactivity and performance

*"A **signal** is a wrapper around a value that notifies consumers when that value changes. Signals can contain any value, from primitives to complex data structures."*

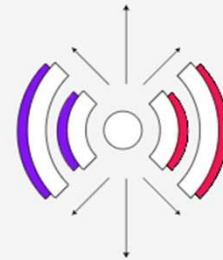
- You read a signal's value by calling its `getter` function, which allows Angular to track where the signal is used.
- Signals may be either *writable* or *read-only*.
- <https://angular.dev/guide/signals>

<https://angular.dev/guide/signals>

In-depth Guides > Signals

Angular Signals

Angular Signals is a system that granularly tracks how and where your state is used throughout an application, allowing the framework to optimize rendering updates.



★ **TIP:** Check out Angular's [Essentials](#) before diving into this comprehensive guide.

Creating and using signals

- In your TypeScript class:

```
// Signal holding the user's name as a string  
protected name = signal<string>('Peter Kassenaar');  
  
// Signal holding the current city as a string  
protected city = signal<string>('Groningen');
```

1. Simple data binding syntax

Double curly braces and 'invoking' with signals:

```
<div>City: {{ city() }}</div>
```

```
<div>First name: {{ person().firstname }}</div>
```


Binding using a loop: @for

- Previously: `*ngFor="let item of items"`
- New: `@for (item of items; track item.id) {...}`
- ONLY: in Angular 17+ (👉 with classic variables)

```
cities : City[] = [  
  {id: 1, name: 'Amsterdam', country: 'NL'},  
  {id: 2, name: 'Berlin', country: 'GER'},  
  {id: 3, name: 'Tokyo', country: 'JAP'},  
]
```

```
<ul>  
  @for (city of cities; track city.id) {  
    <li>{{ city.id }} - {{ city.name }}</li>  
  }  
</ul>
```

- 1 - Amsterdam
- 2 - Berlin
- 3 - Tokyo

Or, using a signal to bind to a collection

```
// Signal containing an array of strings representing cities.  
protected cities = signal<string[]>(['Groningen', 'Hengelo',  
                                     'Den Haag', 'Enschede']);
```

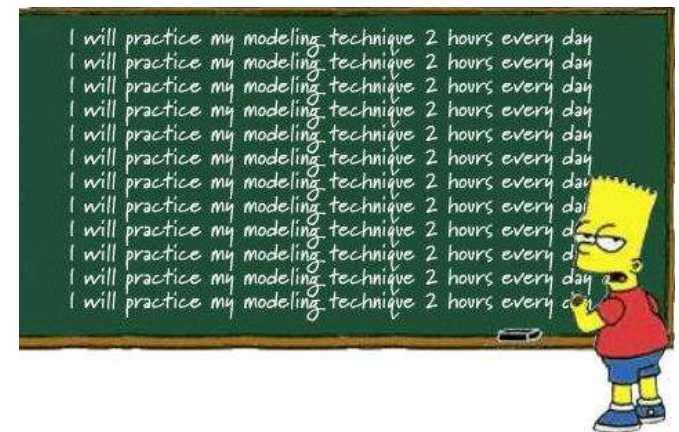
```
<h3>My favourite cities are:</h3>  
<ul>  
  @for (city of cities(); track $index) {  
    <li>{{ city }}</li>  
  }  
</ul>
```

My favourite cities are:

- Groningen
- Hengelo
- Den Haag
- Enschede

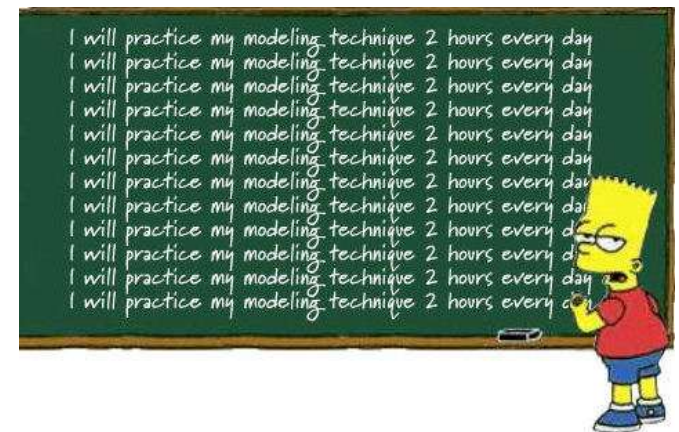
Workshop

- Expand your app from the previous lab (Hello World) with a field/property. Bind the property in the template being used.
 1. Use direct initialization of variables, like `name: string = '<your-name>'`.
 2. Use a signal for different types (`signal<string>`, `signal<number[]>`, ...)
- The user interface will always be the same! The signal approach is just the best practice.
- Use additional properties.
- Demo `../101-databinding`



Workshop

- Create an array of properties. Bind them in the template using the directive `*ngFor` or `@for(...; track ...)`
 - What do you need to import for the first notation?
- Use TypeScript to explicitly declare the property as an array of strings.
 - `cities: WritableSignal<string[]>;` OR
 - `cities: WritableSignal<Array<string>>;`
- Of course you can use other data than cities, for example persons, products, and so on.
- Demo `../101-databinding`.



Checkpoint

- Simple data binding `{{ ... }}`
- **Properties** of the class are bound
- Best practice: wrap properties in **signals**
- You can **bind** them to the template
- Use an **array of data** to bind to the template
 - Use `*ngFor` or `@for()` to loop over data

Creating a Model (as in: MVC)

A Model as a class with exported public properties:

```
export class City{  
  constructor(  
    public id: number,  
    public name: string,  
    public province: string,  
  ){ }  
}
```

Notice shorthand notation `public id : number`

1. Defines a private/local parameter
2. Defines a public parameter with the same name
3. Initializes parameter at instantiation of the class with `new`

Using the Model

1. Import model class

```
import {City} from './city.model'
```

2. Update component

```
export class AppComponent {  
  name = 'Peter Kassenaar';  
  cities = [  
    new City(1, 'Groningen', 'Groningen'),  
    new City(2, 'Hengelo', 'Overijssel'),  
    new City(3, 'Den Haag', 'Zuid-Holland'),  
    new City(4, 'Enschede', 'Overijssel'),  
  ]  
}
```

2a. Or: using signal:

```
protected cities = signal<City[]>([  
  new City(1, 'Groningen', 'Groningen'),  
  new City(2, 'Hengelo', 'Overijssel'),  
  new City(3, 'Den Haag', 'Zuid-Holland'),  
  new City(4, 'Enschede', 'Overijssel'),  
]);
```

3. Update View

```
<ul>  
  @for (city of cities(); track city.id) {  
    <li>{{ city.id }} - {{ city.name }}</li>  
  }  
</ul>
```

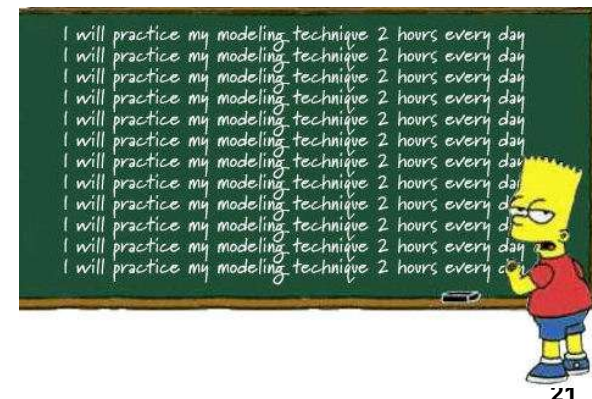
Another option: interface

- Interface only describes the **structure** of the data
- No keyword `new`
- No functionality in the 'instances'
- Mostly – personal preference!

```
interface ICity {  
    id: number;  
    name: string;  
    province: string;  
}
```


Workshop

- Create a **Model for the contents** of your array. The model consists of an object with one or more properties.
 - See for instance `app/shared/city.model.ts` as an example.
- Update the **signature of the array** so it looks like `cities: City[]` or `cities: Array<City>` in your variable or in your `signal<T>`
- **Rewrite** your array, so the content now are properties of type `<YourModel>` (often abbreviated with just `<T>`)
- **Optional:** instead of using a class for the Model you can also use the TypeScript-construct `interface` or `type`.
 - Look up for yourself how this can be done.



Binding conditionally using @if

- Previously: `*ngIf="..."`
- New: `@if (condition) { ... }`
- ONLY : Angular 17+

Invalid

```
<div @if="showTitle">
  {{ title }}
</div>
```

Valid

```
@if (showTitle){
  <div>
    {{ title }}
  </div>
}
```

Classic *ngIf notation

Use the *ngIf directive (pay attention to the asterisk!)

```
<h2 *ngIf="cities.length > 3">There are a lot of favorite cities!</h2>
```



External templates

If you don't like inline HTML :

```
@Component({  
  selector    : 'hello-world',  
  templateUrl: 'app.component.html'  
})
```

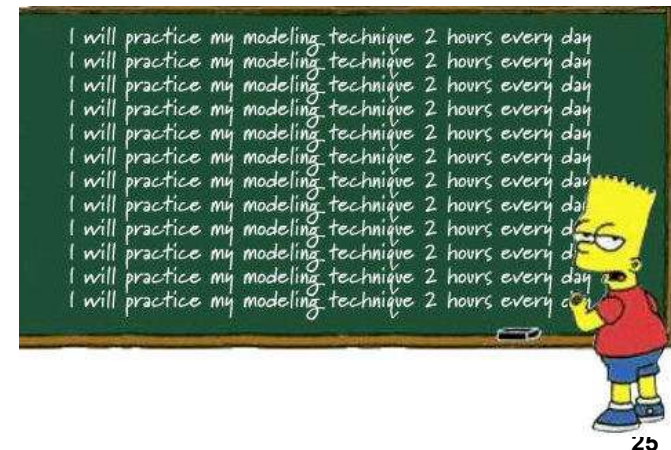


File `app.html`

```
<!-- HTML in external template -->  
<h1>Hello Angular</h1>  
<p>This is an external template</p>  
<h2>My name is : {{ name }}</h2>  
<h2>My favorite cities :</h2>  
...
```

Workshop

- Create a `<div>` on the page that is only shown if your array has three or more objects in it.
 - The code can look like `<div *ngIf="cities.length > 3">...</div>`.
- Create one more use case of `*ngIf` for yourself.
 - Next: use `@if()` modern syntax.
- Move the expression to the TypeScript-part of the application (where it belongs!). Let it evaluate to a true | false value and bind this value via a property in the UI.





User input and event binding

React to mouse, keyboard,
hyperlinks and more

Event binding syntax

Angular: use **parentheses** for capturing and handling events:

Angular 1.x:

```
<div ng-click="handleClick()">...</div>
```

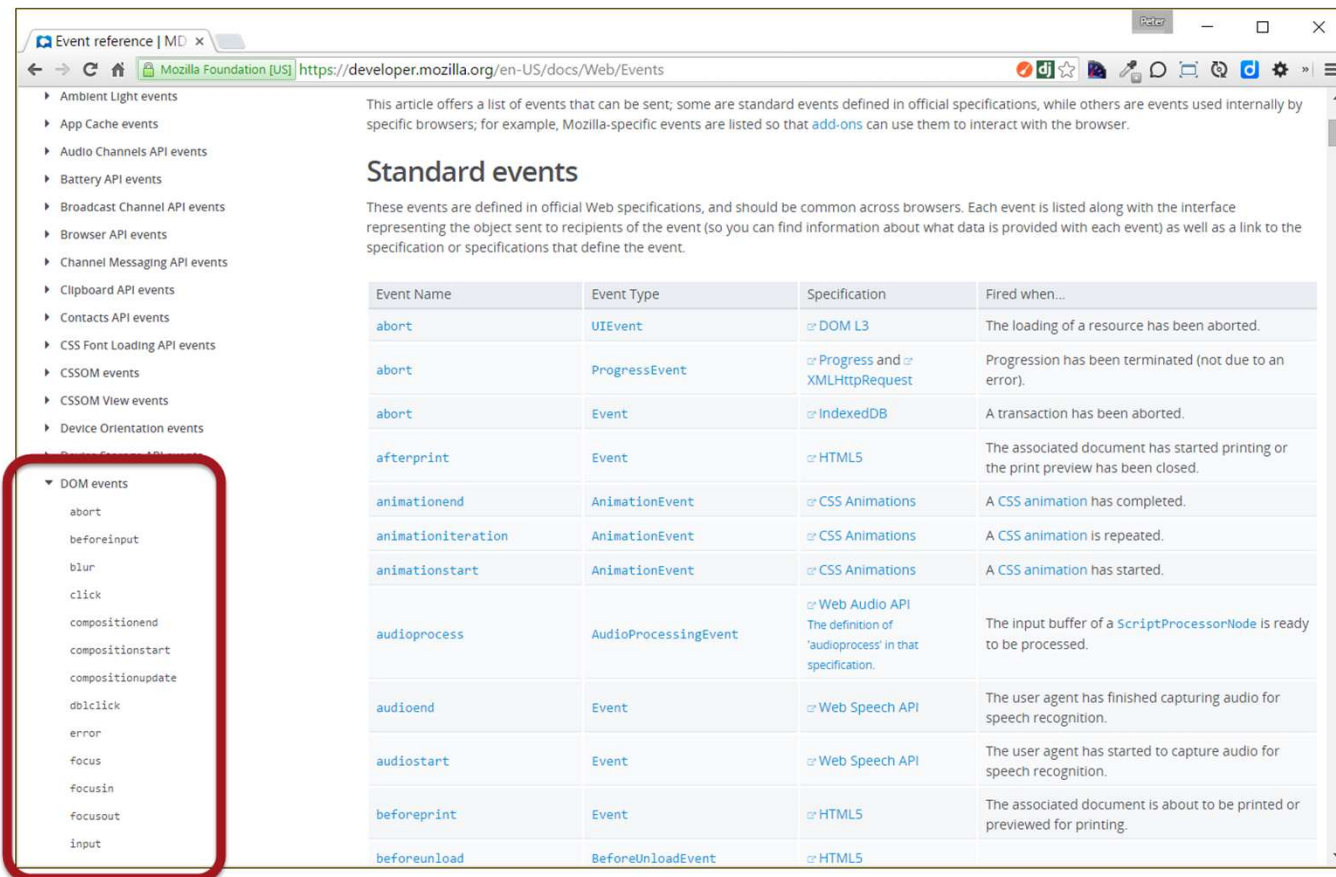
Angular:

```
<div (click)="handleClick()">...</div>
```

```
<input (blur)="onBlur()" />
```

DOM-events

- Angular can listen to *any* DOM-event without needing different directives:



The screenshot shows the MDN 'Event reference' page. The left sidebar lists various event categories, with 'DOM events' highlighted by a red box. The main content area is titled 'Standard events' and contains a table listing various DOM events.

Event Name	Event Type	Specification	Fired when...
abort	UIEvent	DOM L3	The loading of a resource has been aborted.
abort	ProgressEvent	Progress and XMLHttpRequest	Progression has been terminated (not due to an error).
abort	Event	IndexedDB	A transaction has been aborted.
afterprint	Event	HTML5	The associated document has started printing or the print preview has been closed.
animationend	AnimationEvent	CSS Animations	A CSS animation has completed.
animationiteration	AnimationEvent	CSS Animations	A CSS animation is repeated.
animationstart	AnimationEvent	CSS Animations	A CSS animation has started.
audioprocess	AudioProcessingEvent	Web Audio API The definition of 'audioprocess' in that specification.	The input buffer of a ScriptProcessorNode is ready to be processed.
audioend	Event	Web Speech API	The user agent has finished capturing audio for speech recognition.
audiostart	Event	Web Speech API	The user agent has started to capture audio for speech recognition.
beforeprint	Event	HTML5	The associated document is about to be printed or previewed for printing.
beforeunload	BeforeUnloadEvent	HTML5	

<https://developer.mozilla.org/en-US/docs/Web/Events>

Example event binding

HTML

```
<!-- 1. Event binding for a button -->
<button class="btn btn-success"
        (click)="btnClick()">I am a button
</button>
<!-- Current value of the counter signal is written in the UI:-->
<p>You clicked: {{ counter() }} time(s).</p>
<hr>
```

Class

```
protected counter = signal<number>(0);

// 1. Bind to click-event in the page
btnClick() {
  this.counter.update(value => value + 1);
}
```

Remember to call `counter.update()`. It is invalid to update counter directly like `this.counter++`



Invalid!

Result

- Hengelo
- Den Haag
- Enschede

Event binding : mouse and touch

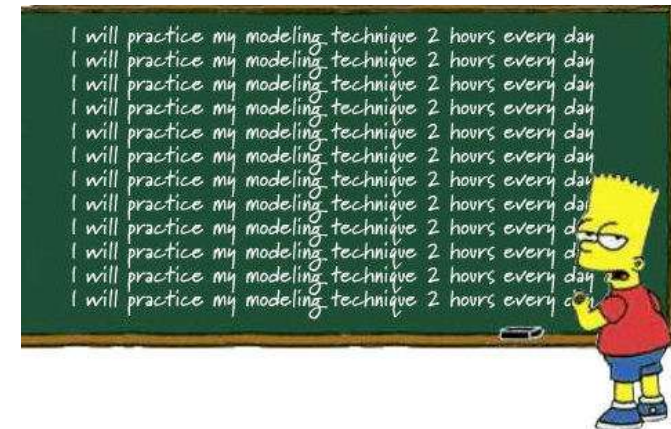
I am a button



You clicked: 4 time(s).

Workshop

- Add an element with event-binding to the application. For instance, create a button to capture a `click` event.
- Call an event handler in the component if the event occurs.
 - For example, write a function `handleClick() { alert('message...') }` or show a `console.log()`-text.
- Next: show the the message via a variable in the UI. It should read something like 'you clicked on <your-entity-name>'.
- Demo .../103-eventbinding..



Checkpoint

- Event binding is done with `(eventname) = "..."`
- Events are always notated in **lowercase**.
- You can bind **multiple events** to the same element.
- Events are ***not* rendered** in the browser DOM-tree
- Events are handled by an event **handler-function** on the component



Reading values from text fields

Creating a variable from your text field

A) Event parameters: \$event

HTML

```
<input type="text" class="input-lg" placeholder="City..."  
      (keyup.enter)="onKeyUp($event)"><br>  
<p>{{ txtKeyUp }}</p>
```

```
// 2. Bind to keyUp-event in the textbox  
onKeyUp(event:any){  
    this.txtKeyUp = event.target.value + ' - ';  
}
```

B) Event parameters local template variable

Declare *local template variable* with # → The complete element is passed to the component

```
<input type="text" class="input-lg" placeholder="City..."
      #txtCity (keyup)="betterKeyUp(txtCity)">
<h3>{{ txtCity.value }}</h3>
```

Class:

```
// 3. Bind to keyUp-event via local template variable
betterKeyUp(txtCity){
  //... Handle txtCity as desired
}
```

Putting it all together...

HTML

```
<input type="text" class="input-lg" placeholder="City..." #txtCity>
<button class="btn btn-success"
  (click)="addCity(txtCity)">Add city
</button>
```

Class

```
export class AppComponent {
  // Properties on component/class
  ...
  addCity(txtCity) {
    let newID    = this.cities.length + 1;
    let newCity = new City(newID, txtCity.value, 'Unknown');
    this.cities.update(newCity);
    txtCity.value = '';
  }
}
```


My favourite cities are:

- Groningen
- Hengelo
- Den Haag
- Enschede
- Maastricht

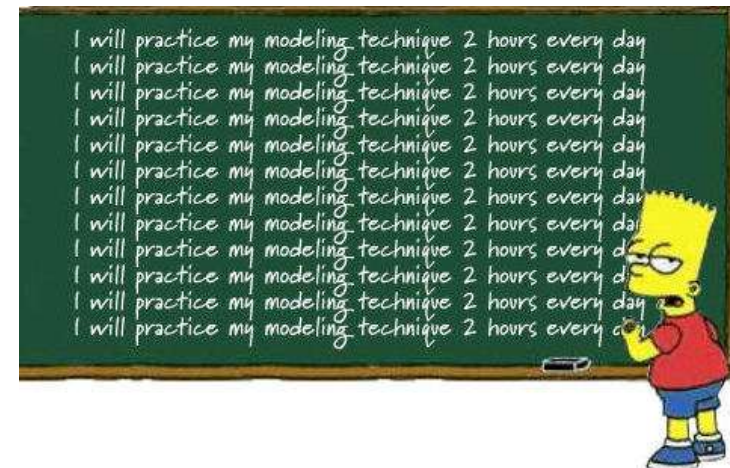
Adding a city to the list



We call *the same* method `addCity()` from `(keyup.enter)` for the textbox, as the `(click)` event for the button.

Workshop

- Create a text field with a **local template variable**.
 - Pass the variable to an **event handler** using an event of your liking (for example click or keyup) and show the value in an `alert()` or `console.log()`
- Create **another textbox** on the page. The user can type numbers in the textbox.
 - Pass the number to an event handler and add the number to a property `total`.
 - Show the addition of all numbers (i.e. the total value) in the page.
- Remember to use `parseInt()` to convert the stringvalue of the textbox to a number!
- Demo .../103-eventbinding.



Checkpoint

- Event binding is addressed with `(eventName) = "..."`
- Events are being handled by a **function** inside the component
- Optional: use `$event` to pass data to the class
- Or: use a **local template variable #** to pass value to the class
- You can create simple, **client sided CRUD-operations** this way.



Attribute & property binding

Bind values dynamically to
HTML attributes and DOM-
properties

Attribute binding syntax

- Bind directly to properties of HTML-elements.
- Also know as *one-way binding*.
- Use square brackets syntax

Angular 1.x:

```
<div ng-hide="true|false">...</div>
```

Angular:

```
<div [hidden]="true">...</div>
```

Or :

```
<div [hidden]="person.hasEmail">...</div>
```

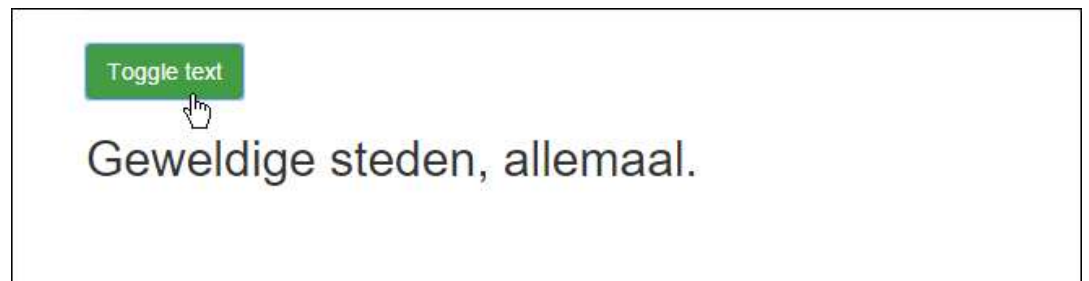
```
<div [style.background-color]=" 'yellow' ">...</div>
```

Example attribute binding

HTML

```
<!-- Attribute binding -->  
<button class="btn btn-success" (click)="toggleText()">Toggle text</button>  
<h2 [hidden]="textVisible">I love all these cities!</h2>
```

```
// Toggle attribute: show or hide text.  
toggleText(){  
  this.textVisible = !this.textVisible;  
}
```



For instance...

HTML

```
@for (city of cities()); track city.id) {  
  <li class="list-group-item"  
    (click)="updateCity(city)">  
    {{ city.id }} - {{ city.name }}  
  </li>  
}
```

Class

```
protected currentCity = signal<City|null>(null);  
protected cityPhoto = signal<string>('');  
  
updateCity(city: City) {  
  console.log(`Updating city: ${city.name}`);  
  this.currentCity.set(city);  
  
  const imageFilename = `assets/img/${city.name}.jpg`;  
  this.cityPhoto.set(imageFilename);  
}
```

Demo:

..\103-attributebinding\src\app\app.ts

My favourite cities are:

1 - Groningen

2 - Hengelo

3 - Den Haag

4 - Enschede



My city: Enschede

More information : <https://angular.dev/guide/templates/binding#css-class-and-style-property-bindings>

Workshop

- Create a button on the page. If the **button is clicked**, a `<div>` with a text is shown.
- If the button is clicked again, the text is hidden.
 - Demo code available at [.../103-attributebinding](#)
- **Optional**: create a component with a textbox.
 - If the user types an English color in the box and clicks a button or presses Enter, a corresponding `<div>` receives this color as a background color.
 - Hint: use `[style.backgroundColor]="bgCcolor"` on the `div`. Create a `bgCcolor` property on the class.
- **Optional**: create a second textbox to set the text/foreground color.
- **Advanced**: investigate how this works if the color can be picked from a series of radio buttons or from a dropdown list.

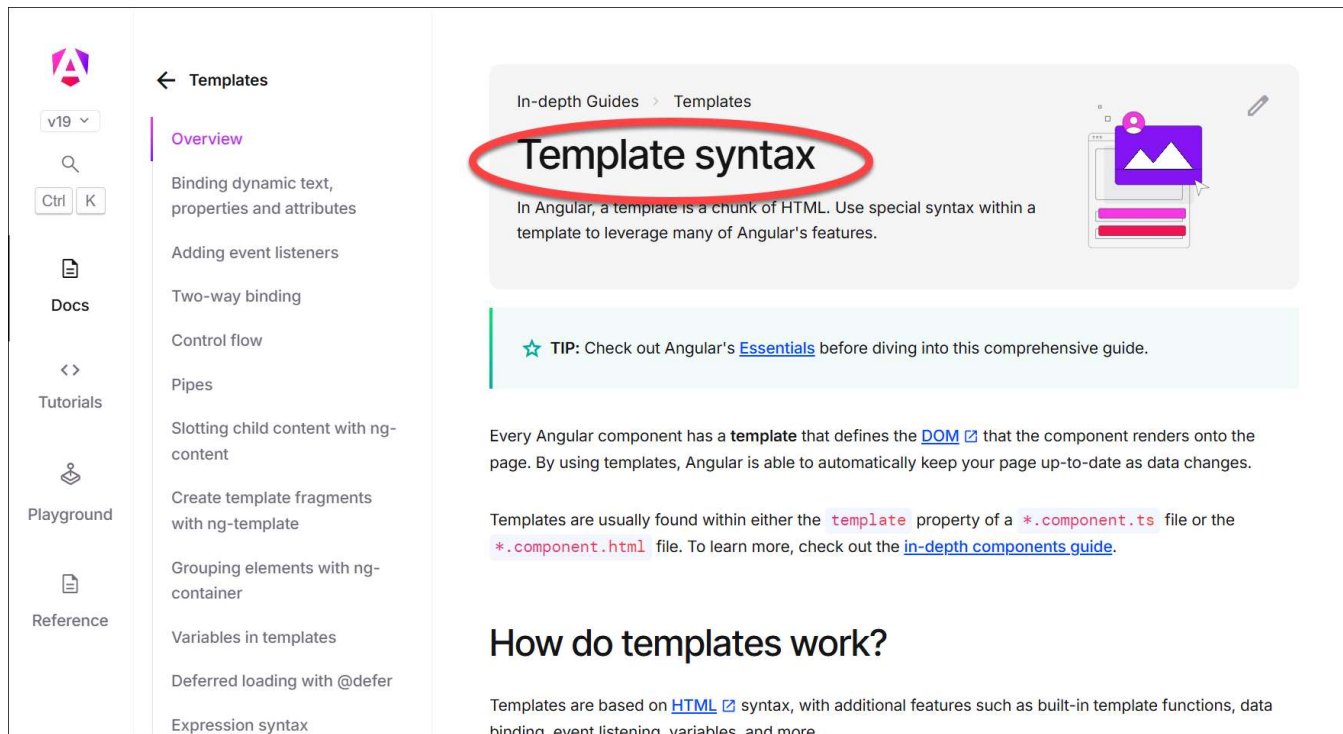


Checkpoint

- Attribute binding is addressed with `[attrName]="..."`
- Attributes are bound to a variable on the class.
- You can calculate the variable in the `.ts`-file

More binding-options

- Attribute binding and DOM-property binding: [...]
- Class binding : [ngClass] (now deprecated)
- Style binding : [ngStyle] (now deprecated!)
- <https://angular.dev/guide/templates>



The screenshot shows the Angular documentation page for 'Template syntax'. The page is divided into three main sections: a left sidebar, a middle navigation pane, and a right content pane. The sidebar contains links to 'v19', 'Docs', 'Tutorials', 'Playground', and 'Reference'. The middle pane lists various topics under the 'Templates' heading, including 'Overview', 'Binding dynamic text, properties and attributes', 'Adding event listeners', 'Two-way binding', 'Control flow', 'Pipes', 'Slotting child content with ng-content', 'Create template fragments with ng-template', 'Grouping elements with ng-container', 'Variables in templates', 'Deferred loading with @defer', and 'Expression syntax'. The right pane is titled 'Template syntax' and contains an introduction to templates, a tip about checking out Angular's Essentials, and a section titled 'How do templates work?'. The 'Template syntax' title is circled in red.

← Templates

Overview

Binding dynamic text, properties and attributes

Adding event listeners

Two-way binding

Control flow

Pipes

Slotting child content with ng-content

Create template fragments with ng-template

Grouping elements with ng-container

Variables in templates

Deferred loading with @defer

Expression syntax

In-depth Guides > Templates

Template syntax

In Angular, a template is a chunk of HTML. Use special syntax within a template to leverage many of Angular's features.

★ TIP: Check out Angular's [Essentials](#) before diving into this comprehensive guide.

Every Angular component has a **template** that defines the [DOM](#) that the component renders onto the page. By using templates, Angular is able to automatically keep your page up-to-date as data changes.

Templates are usually found within either the **template** property of a `*.component.ts` file or the `*.component.html` file. To learn more, check out the [in-depth components guide](#).

How do templates work?

Templates are based on [HTML](#) syntax, with additional features such as built-in template functions, data binding, event listening, variables, and more.



Two-way binding

Updating user interface and
class variables at the same
time

Two way binding syntax

Was removed from Angular for a while, but returned after complaints from the community:

Angular 1.x:

```
<input ng-model="person.firstName" />
```

Angular: similar, but notation is a little bizar:

```
<input [ (ngModel) ]="person.firstName" />
```

Using [(ngModel)]

```
<input type="text" class="input-lg" [(ngModel)]="newCity" />
<h2>{{ newCity }}</h2>
```

Which is shorthand-notation for:

```
<!-- Two-way binding with extended syntax -->
<input type="text" class="input-lg"
      [value]="newCityExtended"
      (input)="newCityExtended = $event.target.value" />
<h2>{{ newCityExtended }}</h2>
```

FormsModule importeren

- Two-way binding used to be in the Angular Core – now in it's own module
 - Import `FormsModule` in your component
 - Classic: import in `app.module.ts`
- `import {FormsModule} from "@angular/forms";`
- ...
- `imports : [FormsModule, ...],`

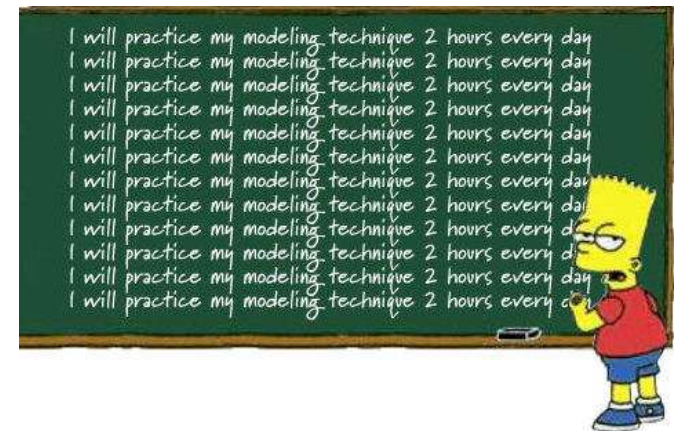
Our Goal: passing data from View to Controller,

We have lots of options:

1. Using `$event`
2. Using a Local Template Variable `#NameVar`
3. Using `[(ngModel)]` (to be used in simple situations, mostly not on complex forms)
4. `HostBinding/@HostListener` (via `@-decorators`)
5. Use `@ViewChild()` ...

Workshop

- Create a text field in your component that uses **two-way binding**.
 - Use `[(ngModel)]` as a directive on the `<input>` box. Bind the value of the typed text directly to the page.
- Create a **copy-function**. Create two text boxes on the page.
 - Text that is typed into the first textbox, should also appear in the second textbox.
- Demo `.../104-twowaybinding`



Checkpoint

- Two-way binding is addressed with `[(ngModel)] = "..."`
- The value of `[(ngModel)]` is updated *automagically* by Angular.
- Use `[(ngModel)]` on `<input type="..." />` tags
- The directive is available in the View/Template and in the TypeScript class.
 - Changes you make in the template are reflected in your class
 - Changes you make in your class are reflected in the template

Declarative syntax

- Four (4) types of databinding
- Angular specific notation in HTML templates
 1. Simple data binding with `{{ ... }}`
 2. Event binding with `( ... )`
 3. One-way data binding (Attribute binding) with `[ ... ]`
 4. Two-way data binding with `[(ngModel)]="..."`

Checkpoint

- **Databinding** in Angular is different from other frameworks
- Learn the **syntax** on DOM- and Attribute binding. Also learn event binding and two-way binding.
- Optional: host binding with `@HostListener()`
- Always **edit** the class and corresponding View
- A lot of **concepts are the same**
 - However, the way to achieve results are different in Angular, compared to Vue, React and other frameworks.